UNIVERSITY OF PADOVA
DEPARTMENT OF INFORMATION ENGINEERING

Biomedical Wearable Technologies
for Healthcare and Wellbeing

Representational State Transfer (REST) and Application Programming Interface (API)

A.Y. 2025-2026

Giacomo Cappon

DEPARTMENT OF INFORMATION ENGINEERING
UNIVERSITY OF PADOVA

Representational State Transfer (REST)

- **Representational state transfer (REST)**: a software **architectural style** created to guide the design and development of the World Wide Web.

- REST defines a set of **constraints** for how the architecture of an Internet-scale distributed hypermedia system (e.g., the Web) should behave.

- Introduced in 2000 by **Roy Fielding** (co-founder of the Apache HTTP Server project) in his doctoral dissertation (University of California, Irvine).

Architectural style

- **Architecture:**
  - how system elements are identified and allocated
  - how they interact to form a system
  - the amount and granularity of communication needed for interaction
  - the interface protocols used for communication.

- **Architectural style:** a set of architectural constraints that restricts the roles and features of architectural elements and the allowed relationships among those elements.

REST: goal and constraints

- REST is a set of architectural constraints.
- REST goal: to **minimize latency and network communication**, while at the same time **maximizing the independence and scalability of components**.
- This is achieved by placing constraints on **the interface between components**, rather than on each component implementation.
- In total, **6 architectural constraints**:
  1. Client-server architecture
  2. Stateless communication
  3. Cache constraint
  4. Uniform interface between components
  5. Layered system
  6. Code-on-demand (optional)

Constraint 1: Client-server architecture

- **Principle of separation of concerns**: a design principle for separating a computer program into distinct sections, in which each section addresses a separate concern (i.e., a different task) → **modular system**

- The **client-server (CS)** architecture relies on the principle of separation of concern: separation of the user interface concerns (client) from the data storage concerns (server)

- Advantages of the client-server architecture
  - It improves the **portability** of the user interface across multiple platforms
  - It improves **scalability** by simplifying the server components
  - It allows the client and the server components to evolve **independently**

Constraint 2: Stateless communication

- Communication must be **stateless** in nature: the server does not store any state about the client session (i.e., the history of requests) on the server-side.
- **Each request** from client to server must be **standalone**
  - It must contain **all information necessary** to understand the request
  - It cannot take advantage of any stored information on the server.
- The **session state** (i.e., the history of requests) is kept entirely **on the client**.
  - If the server needs any information about previous requests to process the current request, then the client must provide this information in the current request.
- Advantages:
  - Improved **visibility**: no need to look beyond the current request to determine its full nature.
  - Improved **reliability**: recovering from partial failures is easier.
  - Improved **scalability**: the server does not have to manage resource usage across requests
- Tradeoff: it may decrease network **performance** by increasing the repetitive data sent in a series of requests

Constraint 3: Cache constraint

- The data within a response to a request must be implicitly or explicitly **labelled as cacheable or noncacheable**.

- If a response is cacheable, a client cache can reuse that response data for later, equivalent, requests.

- Advantage:
  - Improved **efficiency**: it partially or completely **eliminate some interactions.**

- Tradeoff:
  - **Decrease reliability,** if data within the cache differs significantly from the data that would have been obtained with a new request to the server.

Constraint 4: Uniform interface between components

- The information is exchanged in the same way for every client-server interactions, whatever the client and the server are.

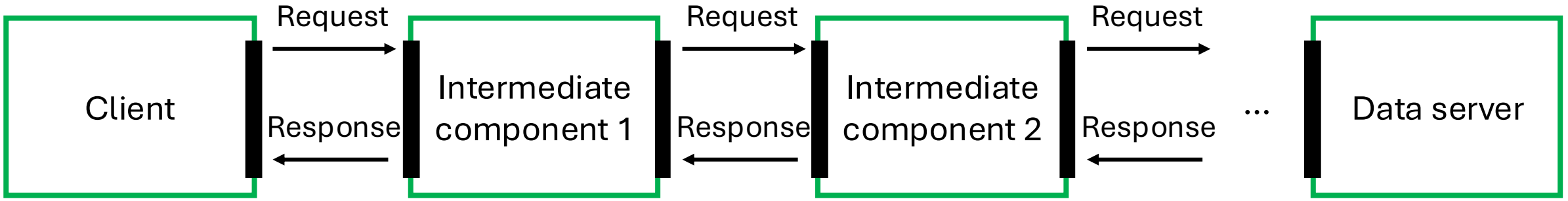
- There is a **common language** between servers and clients that allows each part to be substituted or modified without breaking the entire system.

- Advantage:
  - Independent **evolvability** of the system components: the implementation of interfaces is decoupled from the services they provide.

- Tradeoff:
  - Decreased **efficiency** for some applications: the information is transferred in a standardized form rather than one which is specific to an application's needs.

Constraint 5: Layered system style

- The architect can inject “layers” of service between the server and the client to efficiently serve the client requests.
- The layering must be transparent to the client: each component cannot “see” beyond the immediate layer with which it is interacting → the client does not know if it is talking with final data server or an intermediate server.



- Advantage: by restricting knowledge of the system to a single layer, we place a bound on the overall system complexity and promote component **independence**.
- Tradeoff: layers add **overhead** and latency to the processing of data, reducing user-perceived performance.

Constraint 6: Code-on-demand (optional)

- The client functionalities can be extended by downloading and executing code in the form of applets or scripts.

- Advantage:
  - Clients are **simplified** by reducing the number of features required to be pre-implemented.
  - Allowing features to be downloaded after deployment improves system **extensibility**.

- This constraint is not mandatory but **optional**: the architecture should support this constraint in the general case, but this may be disabled within some contexts.

The definition of the uniform interface

- The REST uniform interface (constraint 4) is defined by putting 4 requirements on the interface between components (clients and servers):

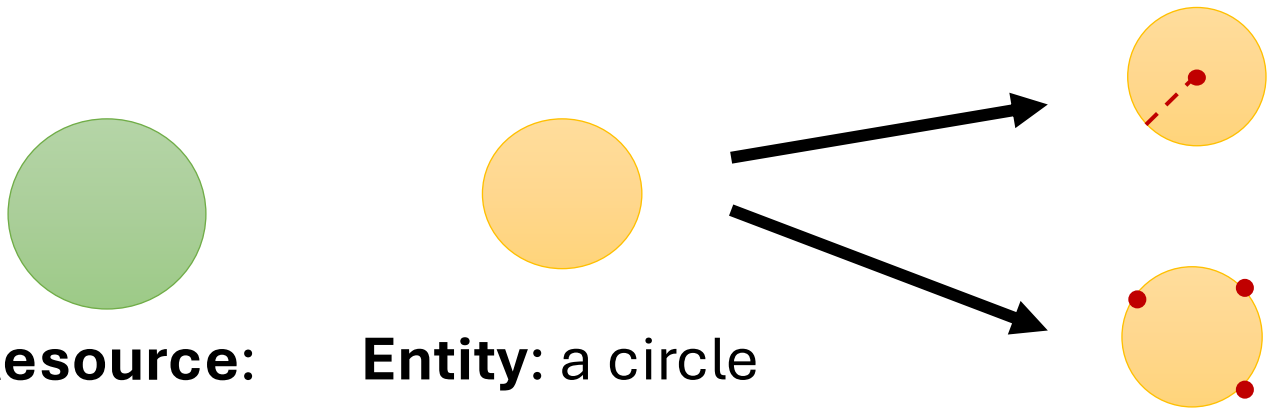
  - Hypermedia as the engine of application state (HATEOAS)
  - Identification of resources
  - Manipulation of resources through representations
  - Self-descriptive messages

Hypermedia as the engine of application state

- Web applications are based on **distributed hypermedia**:
  - When a client interacts with a Web application, the application servers provide information dynamically through **hypermedia.**
  - **Hypermedia:** any content that contains links to some other form of media (e.g., an image, a movie, a text) → generalization of hypertext for a general media.
  - When a link is selected, information is moved from the location where it is stored in the server to the location where it will be used in the client → application state transition.
  - A REST client needs no prior knowledge about how to interact with an application server beyond a generic understanding of hypermedia.

Identification of resources

- **Resource**: any information or concept that can be named and be the target of a hypermedia (a document, an image, a collection of other resources, a temporal service, e.g., “today’s weather in Padova”…).
- More precisely, the resource is the conceptual mapping to an entity or a set of entities.
- The **entity** is the specific realization of the concept identified by the resource.
- The entity to which a resource points is stored and manipulated in a specific **resource representation**
- Each resource is identified by a label called **resource identifier**



Resource: circle

Entity: a circle realization

Representation 1: one point for the circle center and one vector for the circle radius in SVG format

Representation 2: coordinates of three points of the circumference in a csv file

Resources types

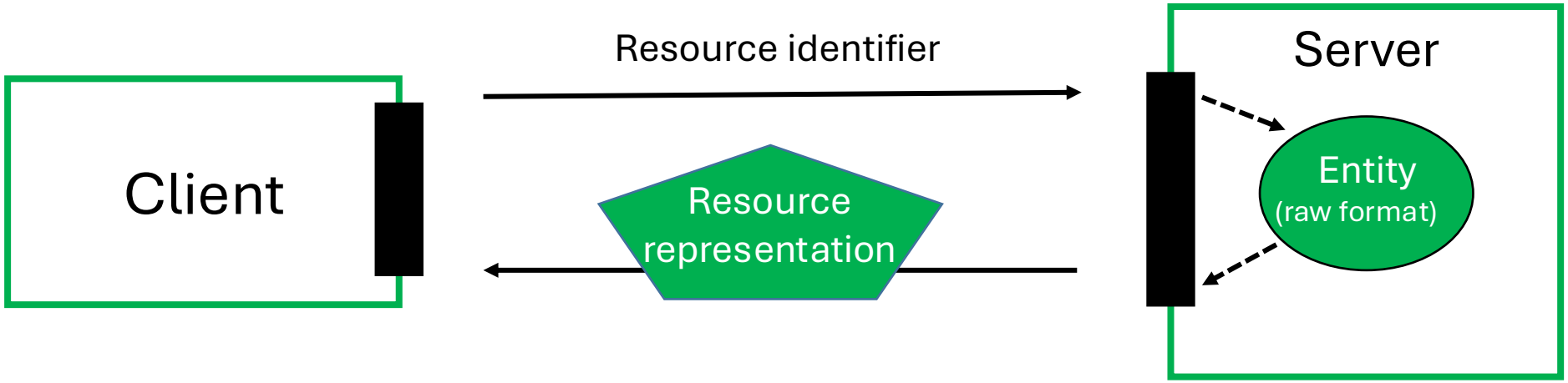
- Example:
  - «the first version of document X»
  - «the revision 1.0 of document X»
  - «the revision 1.1 of document X»
  - …
  - «the latest version of document X»

Static resources: they always point to the same entity (with a specific representation), in this case a precise document version.

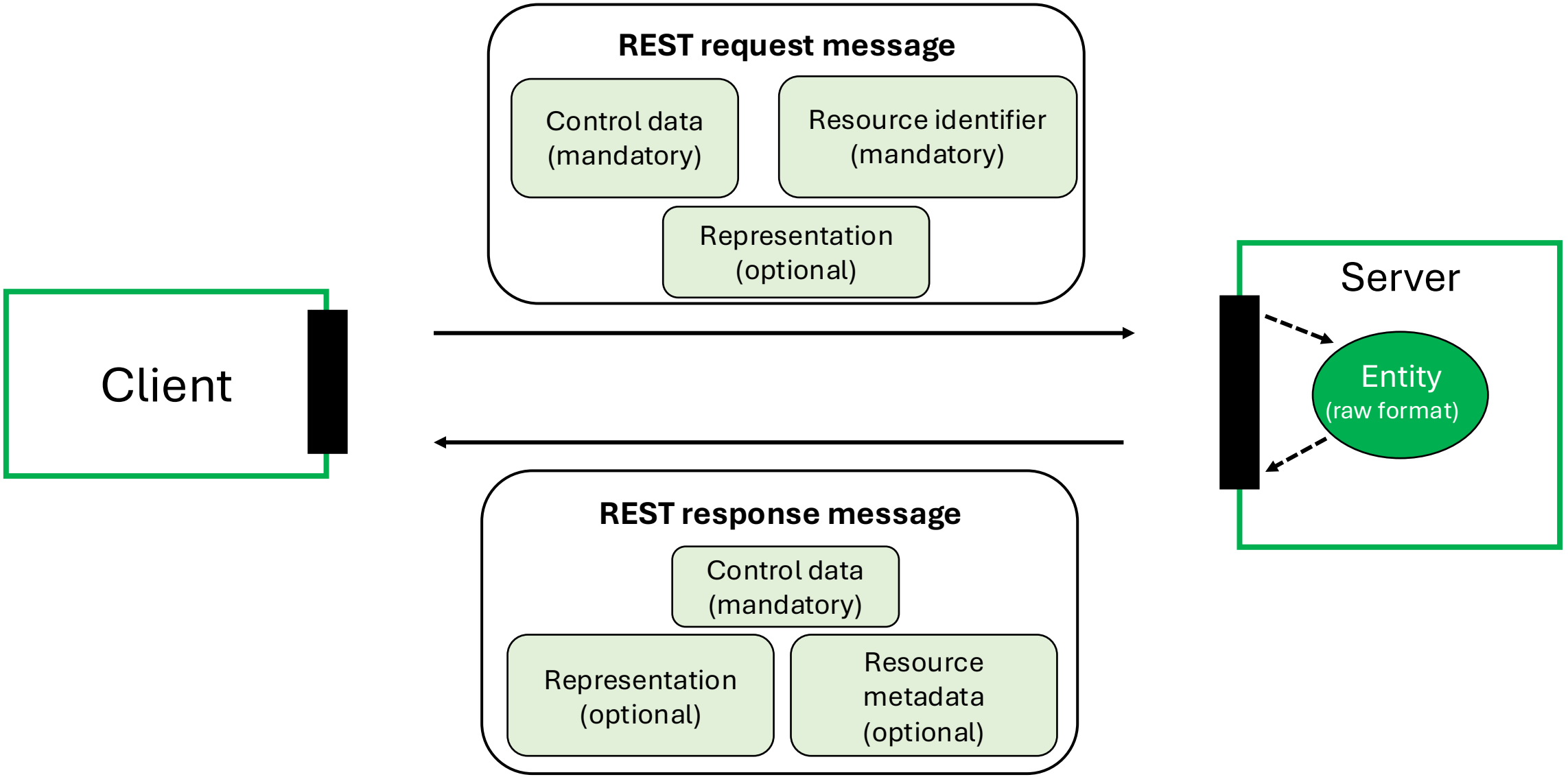
Dynamic resource: the pointed entity can change over time. At some time, the resource «the latest version of document X» will point to the same entity of «the revision 1.1 of document X».

Resource representations

- **Resource representation**: a view of the current state (or realization) of the resource at the time of the request.
- In a client-server interaction, the client asks to the server a resource (by the resource identifier), and the server answers by sending the corresponding resource representation.
- **Note**: The representation sent to the client is not necessarily the same used to store the data in the server.



REST self-descriptive messages



Resource representations and REST messages

- A resource **representation** includes:
  - **Data** (the actual representation)
  - **Metadata** (description of the representation) → typically, not displayed to the end user (e.g., data type, creation date, version number, …)
- The resource representation is provided to the client within a **response message** that contains some additional contents:
  - **Resource metadata (optional)**: additional information about the resource that exists on the server (e.g., alternate text to be displayed in the event an image representation cannot be displayed for some reason).
  - **Control data (mandatory)**: it deals with the validity of the resource and its representation on the client (e.g., if data is cacheable, the expiry time, a checksum for checking integrity, …).

Summary of REST

- An architectural style for **distributed hypermedia** systems
- 6 constraints
  - **Client-server** architecture
  - **Stateless** communication
  - **Cache** constraint
  - **Uniform interface**
  - **Layered** system
  - **Code-on-demand** (optional)

- The uniform interface is realized through the definition of abstract **resources**
  - Each resource is identified by a **resource identifier**
  - The realization of a resource is an **entity**
  - A resource is manipulated by resource **representations** that reflect the current realization of the resource
  - Representations can be sent from server to client (or viceversa) encapsulating them in a **self-explained message**
  - A request message must include **control data** (mandatory), resource identifier (mandatory) and representation's data and metadata (optional)
  - A response message must include control data (mandatory), representation's data and metadata (optional) and **resource metadata** (optional)

The REST paradigm increases performance, scalability, simplicity, modifiability, visibility, portability, and reliability of the system.

Application programming interface (API)

- Two separate applications need an **intermediary** to talk to each other and to allow one application to access the information or functionality of the other.

- **Application programming interface (API):** a connection or interface between computer programs, which **defines how** the systems can interact.
  - The kind of requests that can be made to a system
  - How to make them
  - The data formats that should be used
  - The conventions to follow

- In contrast to a user interface, which connects a computer to a person, the **API connects pieces of software**

Application programming interface (API)

- Why do we need API? To allow **interoperability**, i.e., to allow multiple systems to cooperate.

- An API defines how two software systems can interact while hiding the internal details of how each system works. The API exposes only those parts that are needed for the systems to interact and keeps them consistent even if the internal details of the systems later change → **Modular programming**

- The API is often made up of different tools or services that other applications can call.

- **API specification:** a document or standard describing how to build or use the API tools or services.

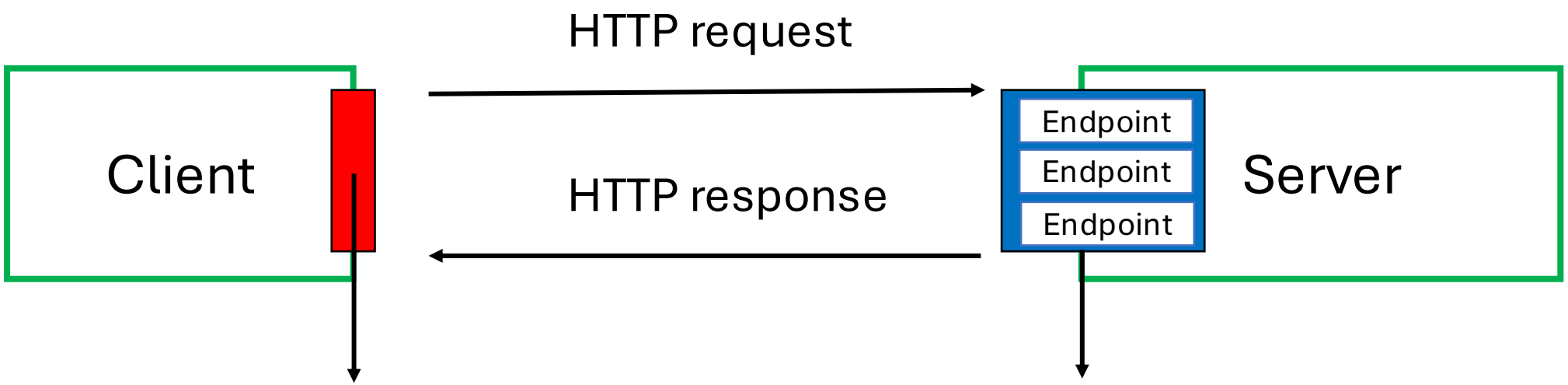
- An API may be **custom-built** for a particular pair of systems, or it may be a **shared standard** allowing interoperability among many systems.

Web API

- **Web API**: an interface for clients and servers that allows them to interact through the Web.
- **Server-side Web API**: the interface of a Web server towards the Web. It consists of one or more publicly exposed **endpoints**.
  - Endpoints: the URLs which specify where the resources that can be accessed by clients lie.
- **Client-side Web API**: a software interface that a client uses to send requests to a server-side Web API.
  - It allows a client web browser or web application to extend its functionality, by exploiting resources (data, programs, more in general services) provided by Web servers.

Request-response Web API

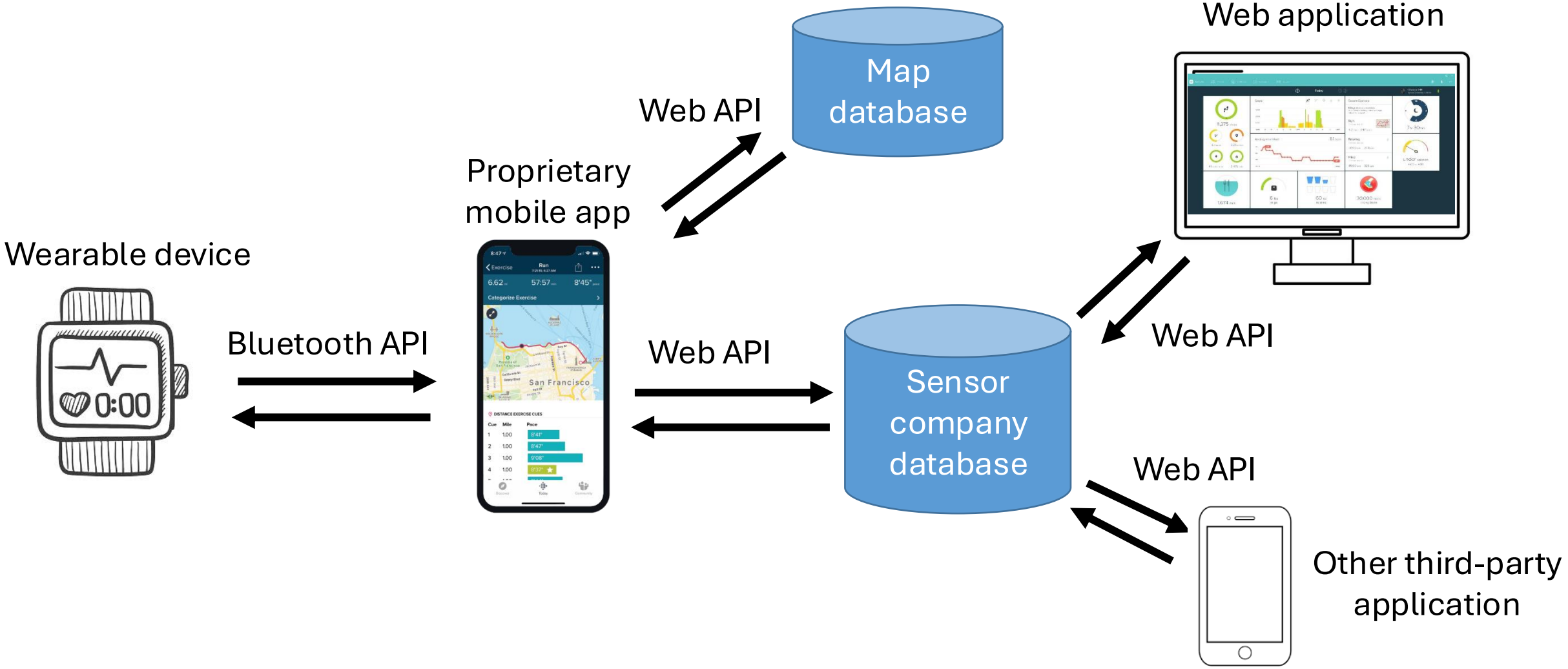
- Most of Web APIs are request-response APIs that rely on the HTTP protocol.



Client-side API:
The piece of software that sends requests to the server.

Server-side API:
The piece of software that receives the requests and the endpoints that provide the requested resources.

Examples of API in wearable sensors



Approaches to design request-response Web API

- Web APIs are realized using protocols and/or specifications to define the semantics and syntax of the messages exchanged by client and server.

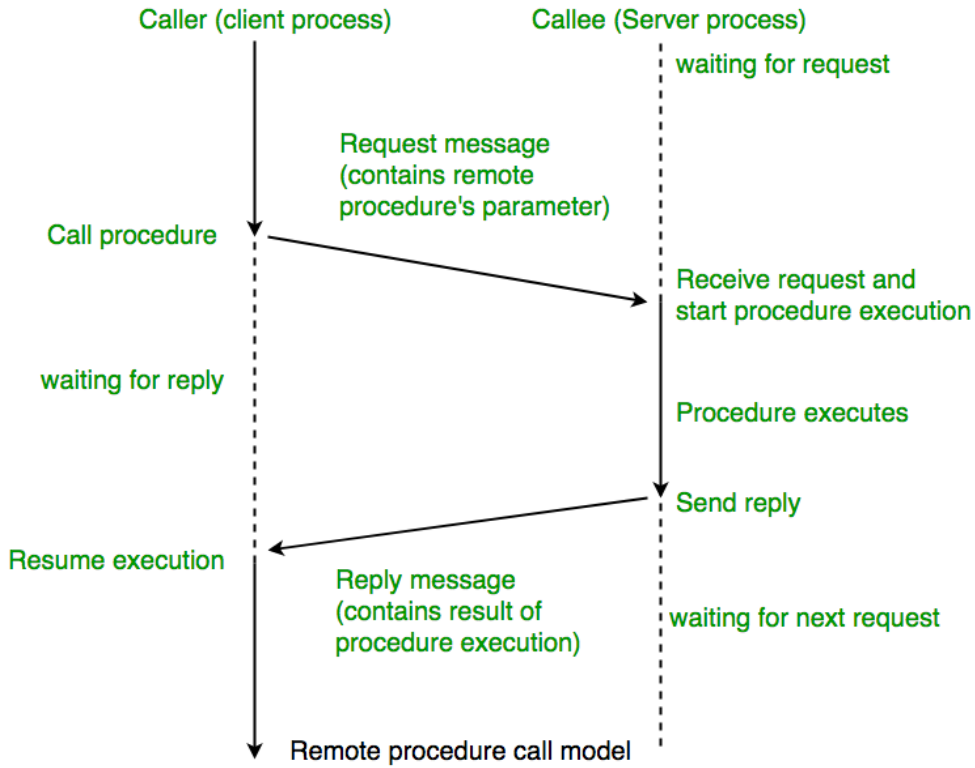
  → **API architectures**

- Most popular request-response API architectures:
  - **RPC**: Remote Procedure Call
  - **SOAP**: Simple Object Access Protocol
  - **REST** architectural style
    - In this case the API is called **RESTful API**

Remote Procedure Call (RPC)

- The simpler and oldest web API approach
- **Remote Procedure Call (RPC)** is a request-response protocol by which the client sends a request message to a known remote server to execute a specified procedure with supplied parameters.

- RPC extends the conventional local procedure calling so that the called procedure does not need to be in the same address space as the calling procedure. The two processes may be on the same system, or on different systems with a network connecting them.
- In RPC **the endpoints represent actions**



Remote Procedure Call (RPC)

- Pros:
  - **Straightforward and simple interaction:** all is done with GET and POST HTTP methods
  - **Easy to add functions**: a new function can be defined and associated to a new endpoint
  - **High performance**: having the functions in the URL allows to have lightweight payloads

- Cons:
  - **Tight coupling to the underlying system:** RPC messages are not standardized. Their format depends on the specific application → low reusability of the API

- Use cases: to perform extremely high-performance, low-overhead internal messaging within big companies (Google, Facebook, Twitch, etc.).

Example of RPC

- This example illustrates an online shop system that tries to handle a transaction with RPC.
- The function paymentService.prcessPayment is implemented in another system.
- User must know a lot of things to use the function: i.e., how to create a Card, function parameters, the structure of a Result, etc…

```
Card card = Card();
card.setCardNumber("1234 5678 1234 5678");
card.setExpiryDate("11/30");
card.setCVC("123");

Result result = await paymentService.processPayment(card,
3.99, Currency.EUR);

if(results.isSuccess()){
        fulfilOrder();
}
```

Simple Object Access Protocol (SOAP)

- Released by Microsoft in 1998
- Web API protocol with highly standardized messages written in XML
- It runs over HTTP or other application layer protocol

```
<?xml version="1.0"?>

<soap:Envelope
xmlns:soap="http://www.w3.org/2003/05/soap-envelope/"
soap:encodingStyle="http://www.w3.org/2003/05/soap-encoding">

<soap:Header>
...
</soap:Header>

<soap:Body>
...
  <soap:Fault>
  ...
  </soap:Fault>
</soap:Body>

</soap:Envelope>
```

It defines the XML document as a SOAP message

Application-specific information specifying how the recipient should process the message (optional)

The message body

Error messages (optional)

Example of SOAP messages

Example of SOAP request:

```
POST /InStock HTTP/1.1
Host: www.example.org
Content-Type: application/soap+xml; charset=utf-8
Content-Length: nnn

<?xml version="1.0"?>

<soap:Envelope
xmlns:soap="http://www.w3.org/2003/05/soap-envelope/"
soap:encodingStyle="http://www.w3.org/2003/05/soap-
encoding">

<soap:Body xmlns:m="http://www.example.org/stock">
  <m:GetStockPrice>
    <m:StockName>IBM</m:StockName>
  </m:GetStockPrice>
</soap:Body>

</soap:Envelope>
```

Example of SOAP response:

```
HTTP/1.1 200 OK
Content-Type: application/soap+xml;
charset=utf-8
Content-Length: nnn

<?xml version="1.0"?>

<soap:Envelope
xmlns:soap="http://www.w3.org/2003/05/soap-
envelope/"
soap:encodingStyle="http://www.w3.org/2003/0
5/soap-encoding">

<soap:Body xmlns:m="http://www.example.org/s
tock">
  <m:GetStockPriceResponse>
    <m:Price>34.5</m:Price>
  </m:GetStockPriceResponse>
</soap:Body>

</soap:Envelope>
```

SOAP: pros and cons

- Pros:
  - Web Services Security (**WS-Security**): an extention of SOAP that allows to use security mechanisms (e.g., signature of messages, encryption of messages, authentication)
  - SOAP allows to perform **stateful communications**, necessary for complex transactions in which multiple parties are involved (e.g., online banking and e-commerce applications)

- Cons:
  - Only XML
  - Heavy messages
  - Specialized knowledge of the protocol rules needed

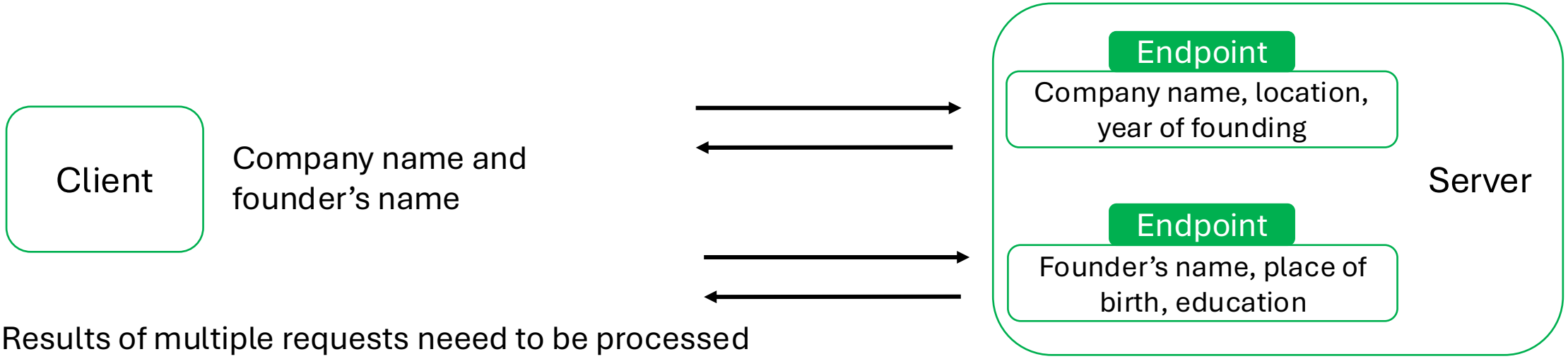
RESTful APIs

- With the expansion of modern Web, a more simple, flexible and scalable way of defining API was needed.
- **RESTful APIs**: APIs designed according to the REST principles, proposed in 2000 by Roy Fielding.
- REST is resource-based (instead of action-based) and it only uses the HTTP methods (GET, POST, PUT, DELETE) → no specialized knowledge required
- No strict protocol, only compliance with the 6 REST constraints
- No constraints on the message format → **JSON** preferred to XML

RESTful APIs: pros and cons

- Pros:
  - Easy, no specialized knowledge required
  - Multiple formats supported
  - Great flexibility and scalability
  - Cache-friendly

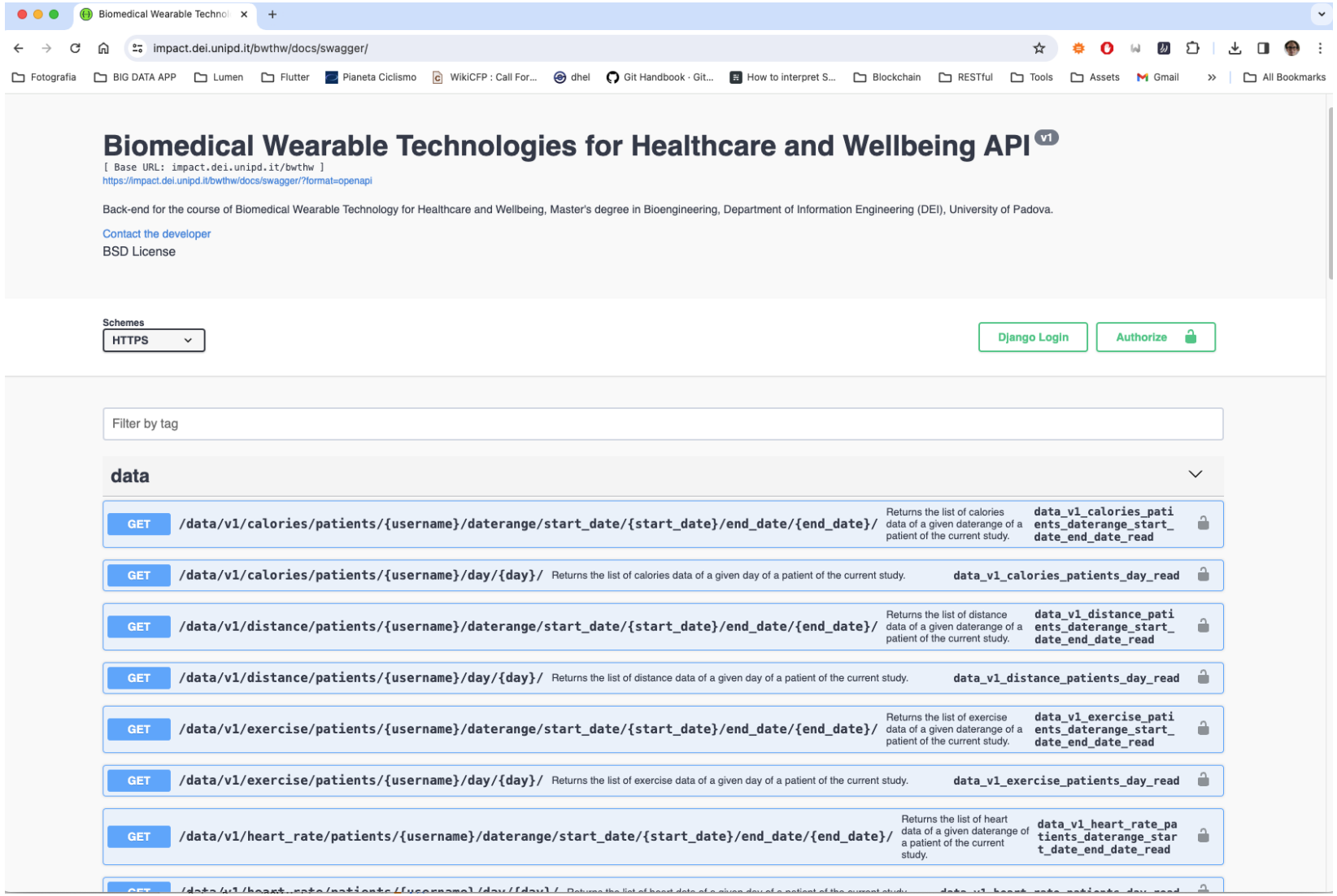
- Cons:
  - REST messages have large payloads to transfer metadata necessary to understand the state of the application and to enable complex actions.
  - Chatty interactions: it generally requires multiple requests to get the needed data. Often unnecessary data are transferred.



Results of multiple requests neeed to be processed (combined and filtered) on client side.

The IMPACT BWTHW RESTful API

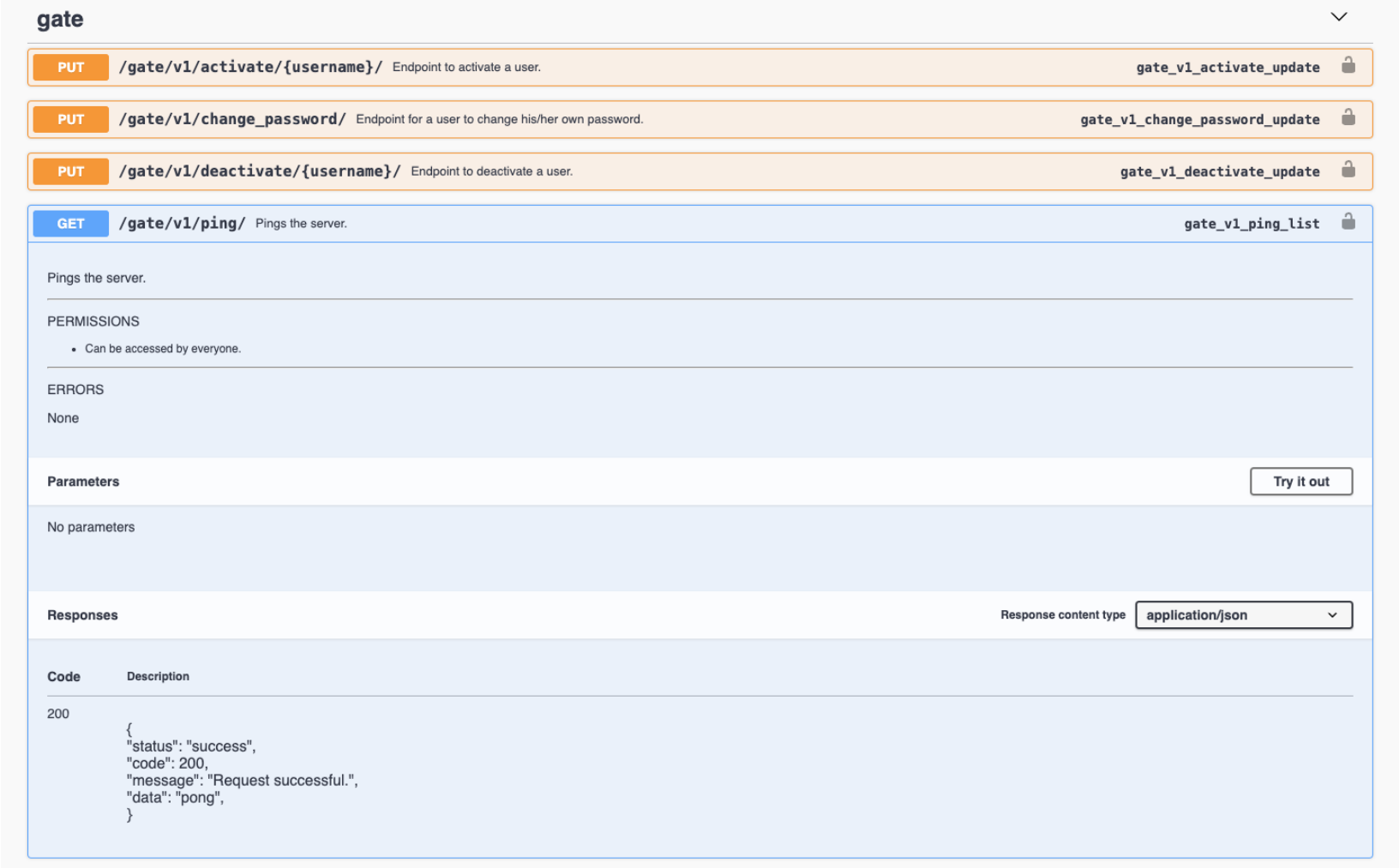
- This is the API you will use to build your project
- It allows to authenticate, ping (to know if the server is alive), and to get data
- Full URL: https://impact.dei.unipd.it/bwthw/



The ping endpoint

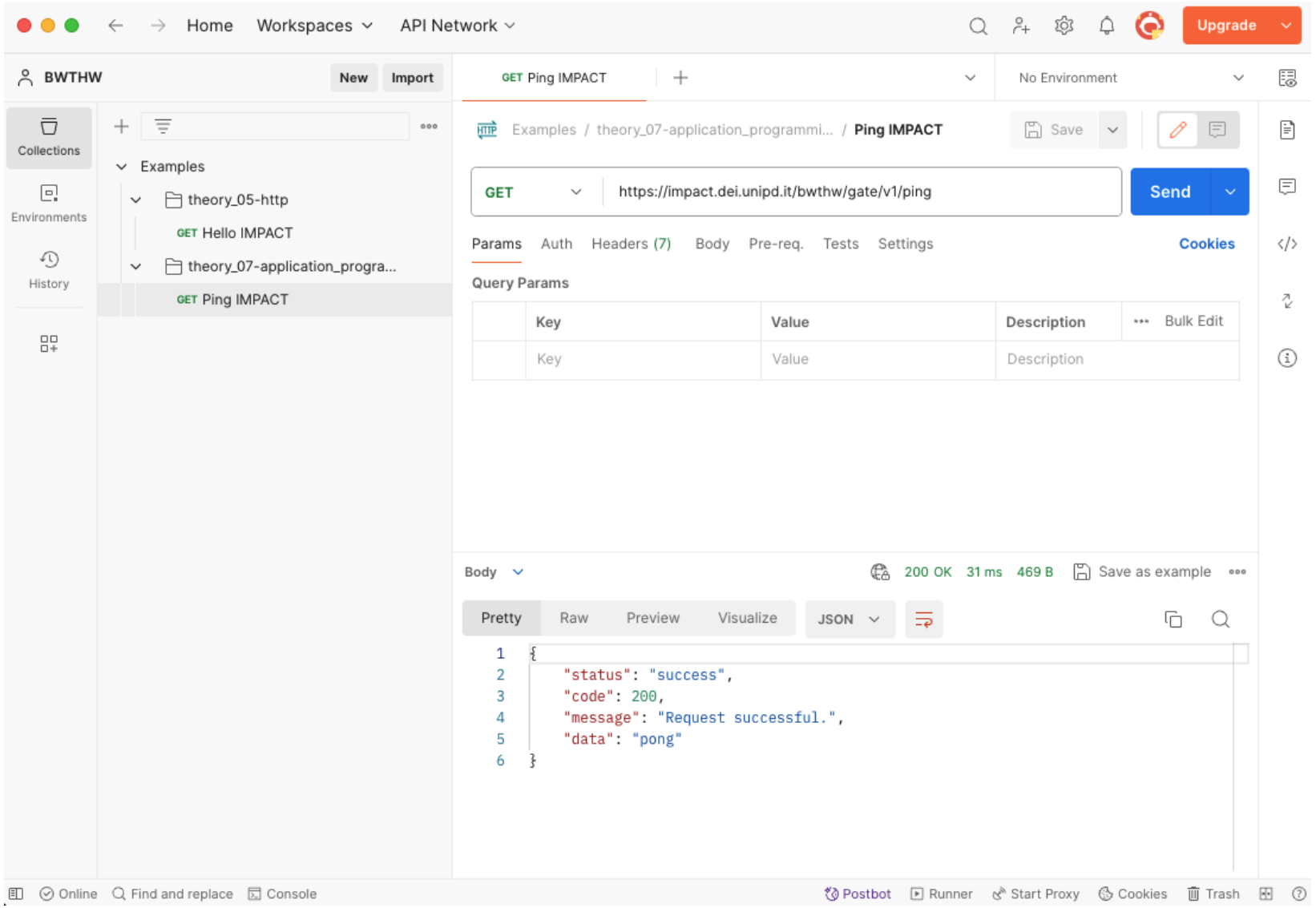
- Let's inspect the **ping** endpoint

- Key things we can learn from the docs:
  - Method: GET
  - Full URL: https://impact.dei.unipd.it/gate/v1/ping/
  - It does not require parameters
  - It does not require to be authenticated
  - If successful, the response will be a JSON



The ping endpoint

- POSTMAN can be used to make a request and test it

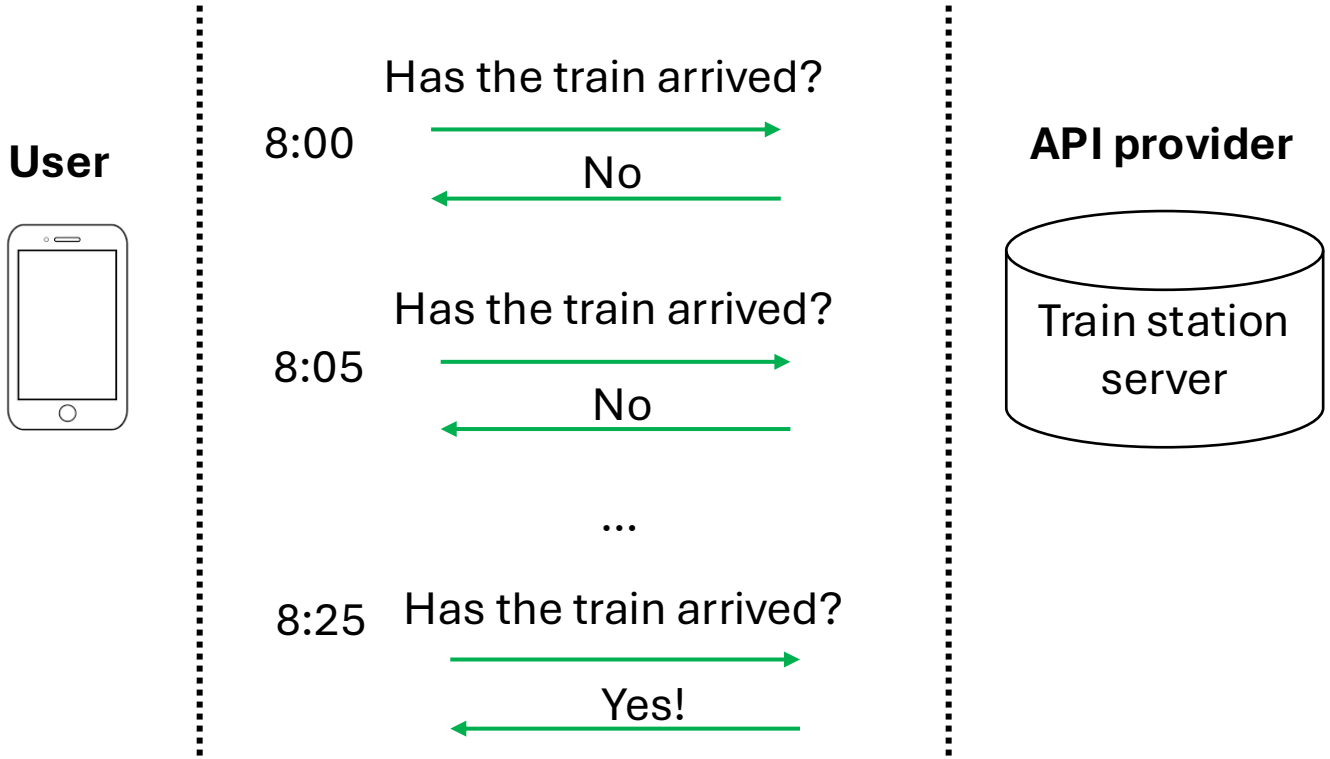


Summary of types of request-response Web APIs

	RPC	SOAP	RESTful
Organized in terms of	Local procedure calling	Enveloped message structure	Compliance with 6 architectural constraints
Format	Multiple (JSON, XML, Protobuf, ...)	XML only	Multiple (JSON, XML, HTML, plain text, ...)
Statefull or stateless	Both	Both	Stateless
Learning curve	Easy	Difficult	Easy
Community	Large	Small	Large
Use cases	Command and action-oriented APIs, internal high performance and massive micro-services	Payment gateways, identity management, financial applications	Public APIs, simple resource-driven app

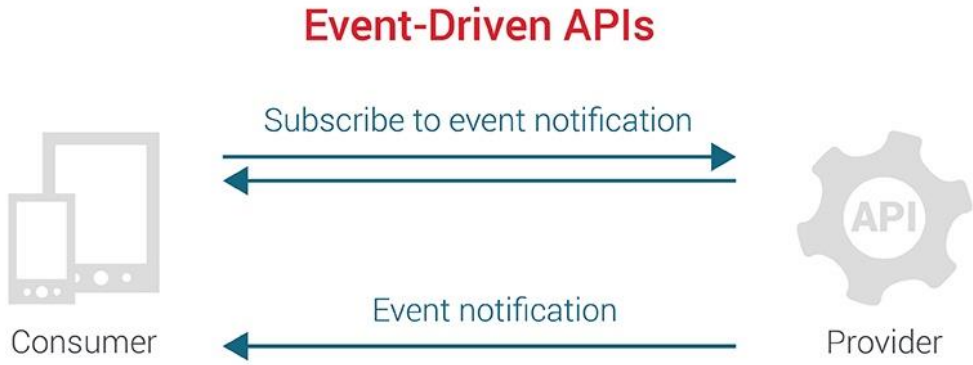
Request-response API architecture

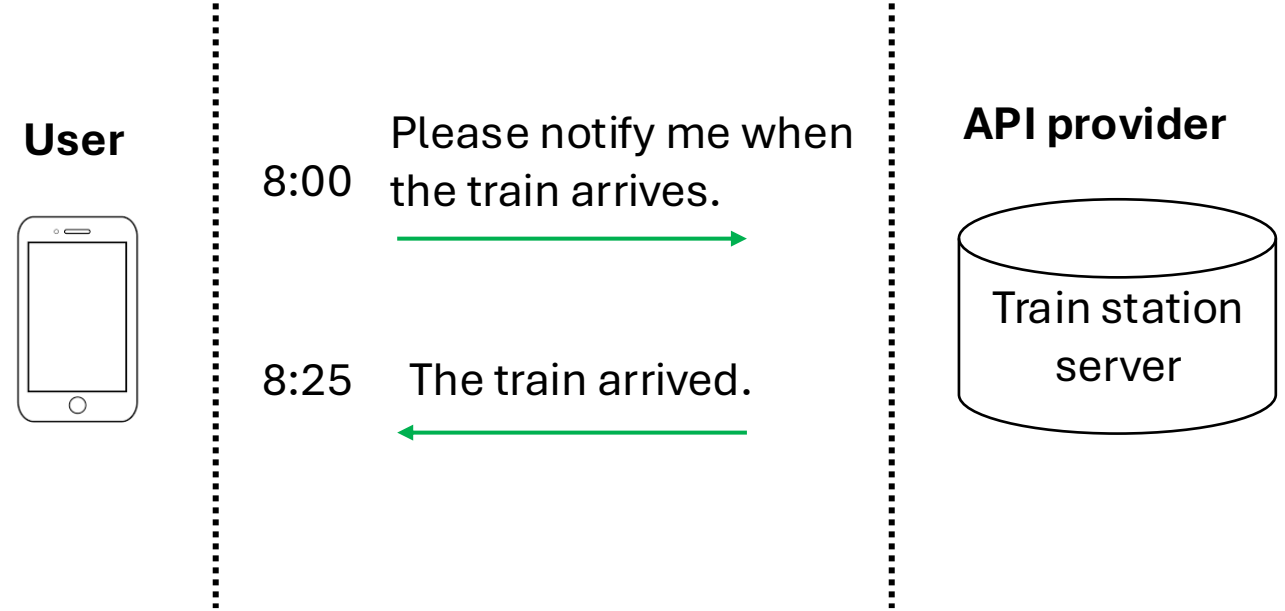
- RPC, SOAP, RESTful, GraphQL are all types of **request-response Web API**
- To get the latest information, the client always have to **poll** the API provider (server).



Event-driven API architecture

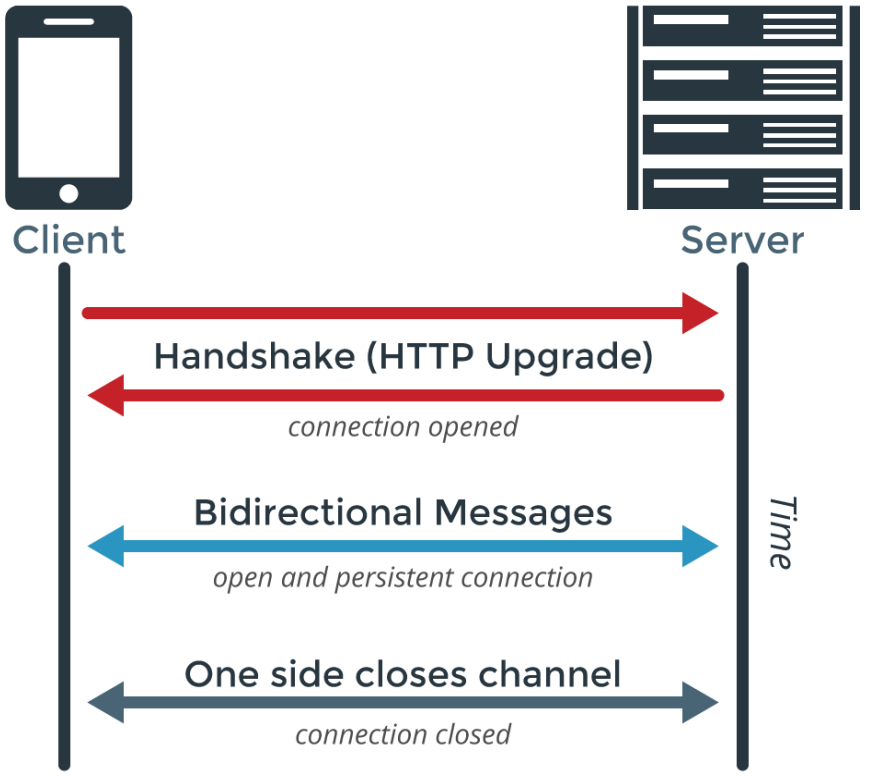
- In **event-driven APIs** the API provider notifies the client when something interesting happens. Notifications do not need a request from the client.
- An event-driven API must offer two capabilities to its clients
  - A mechanism to allow clients to subscribe to events of their interest.
  - Delivery of events to subscribed clients in an asynchronous manner.





WebSocket

- An example of protocol for event-driven APIs
- **WebSocket** allows for constant, bi-directional communication between the server and client, which means both parties can communicate and exchange data as and when needed.
- Suitable for realizing Web chats.



References

- Roy T. Fielding and Richard N. Taylor, “Principles and design of the modern Web architecture”, ACM Transactions on Internet Technology, Vol. 2, No. 2, May 2002, Pages 115–150. https://www.ics.uci.edu/~taylor/documents/2002-REST-TOIT.pdf